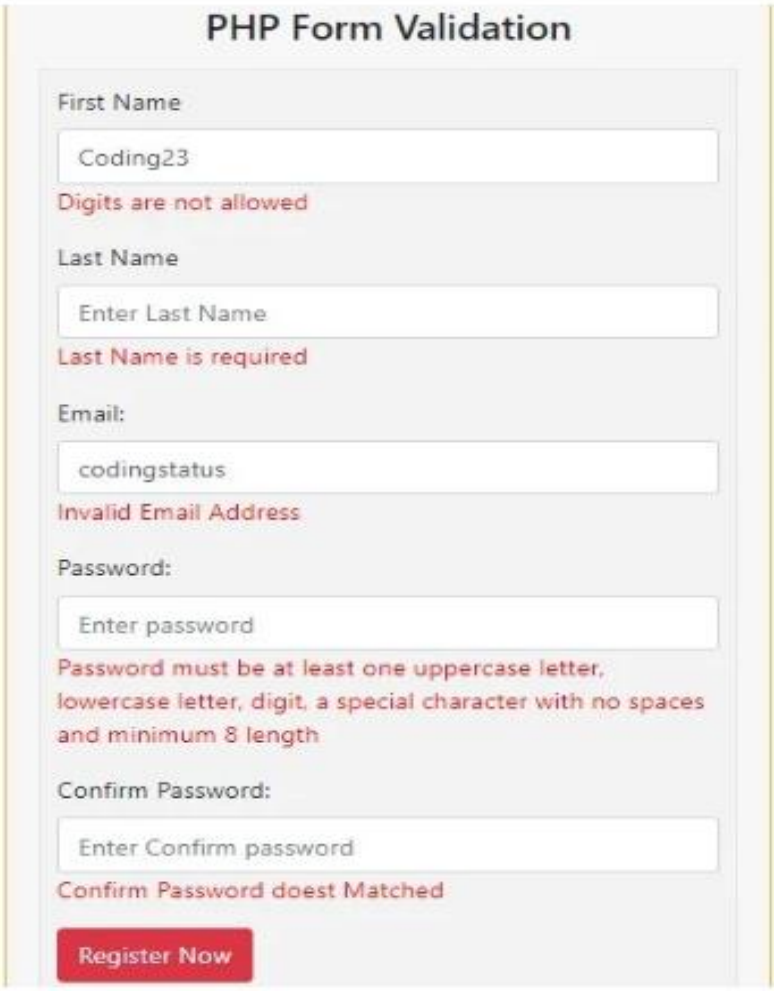
How to Validate Form Data in PHP?

- What is Validation?
  - Validation means check the input submitted by the user. There are two types of validation are available in PHP. They are as follows −
    - Client-Side Validation − *Validation is performed on the client machine web browsers.*
    - Server Side Validation − *After submitted by data, the data has sent to a server and perform validation checks in server machine.*
- PHP Form Validation is the backend or server-side validation. It prevents from entering invalid data into the input field.

Use of Validation:

- It prevents a user's unauthorized access.
- It helps provide genuine data.
- Client side validation saves servers from unwanted and unnecessary load which may otherwise lead to its crash.
- Validation prevents the database from being overloaded due to anonymous storage of information.
- Security can be implemented easily with validation.
- Validation done using PHP cannot be breached by the user since it is done with the server-side validation.



How to Validate Form Using PHP

Before validating the form, you have to set the following basic requirements.

Create Form Validation Rules that are required in your project form

1. All the Input Fields must be required.
2. First Name & Second name must accept only characters with a single white space
3. Email Address must be valid format containing @ symbol.
4. The password must contain a combination of one uppercase & lowercase letter, number, special characters & minimum characters' length 8. Even It will not accept any white spaces. Confirm password must be matched with a password.

Some of Validation rules for field

Field	Validation Rules
Name	Should required letters and white-spaces
Email	Should required @
Website	Should required a valid URL
Radio	Must be selectable at least once
Check Box	Must be checkable at least once
Drop Down menu	Must be selectable at least once

To validate form data in PHP, we can utilize a combination of sanitization and validation techniques. By accessing form input through PHP's superglobal arrays like $_POST or $_GET', we can sanitize the data using functions like filter_var( ) to remove malicious characters. Subsequently, we validate the input against specific formats or constraints, ensuring accuracy and security in web applications.

1. Create A Folder Structure

You should also create the following folder structure

```
myform/
  |__script.php
  |__form.php
  |
```

2. Create An HTML Form

This PHP script creates an HTML registration form with Bootstrap styling, includes a validation script, and handles form submissions, displaying validation errors and success messages.

File Name – form.php

```php
<?php
include('script.php');
?>
<!DOCTYPE html>
<html lang="en">
<head>
<title>PHP Form Validation</title>
<meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<!--bootstrap4 library linked-->
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
</head>
<body>
<div class="container-fluid">
<div class="row">
<div class="col-sm-4">
</div>
<div class="col-sm-4">
<!--=====registration form=====-->
<div class="registration-form">
<h4 class="text-center">Create a New Account</h4>
<p class="text-success text-center"><?php echo $valid; ?></p> <form action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>" method="post">
<!--//first name//-->
<div class="form-group">
<label for="email">First Name</label>
<input type="text" class="form-control" placeholder="Enter First Name" name="first_name" value="<?php echo
$set_firstName;?>">
<p class="err-msg">
<?php if($fnameErr!=1){ echo $fnameErr; }?>
</p>
</div>
<!--//Last name//-->
<div class="form-group">
<label for="email">Last Name</label>
<input type="text" class="form-control" placeholder="Enter Last Name" name="last_name" value="<?php echo
$set_lastName;?>">
<p class="err-msg">
<?php if($lnameErr!=1){ echo $lnameErr; } ?>
</p>
```

```
</div>
<!--// Email//-->
<div class="form-group">
<label for="email">Email:</label>
<input type="text" class="form-control" id="email" placeholder="Enter email" name="email" value="<?php echo
$set_email;?>">
<p class="err-msg">
<?php if($emailErr!=1){ echo $emailErr; } ?>
</p>
</div>
<!--//Password//-->
<div class="form-group">
<label for="pwd">Password:</label>
<input type="password" class="form-control" placeholder="Enter password" name="password">
<p class="err-msg">
<?php if($passErr!=1){ echo $passErr; } ?>
</p>
</div>
<!--//Confirm Password//-->
<div class="form-group">
<label for="pwd">Confirm Password:</label>
<input type="password" class="form-control" placeholder="Enter Confirm password" name="cpassword">
<p class="err-msg">
<?php if($cpassErr!=1){ echo $cpassErr; } ?>
</p>
</div>
<button type="submit" class="btn btn-danger" value="Register" name="register">Register Now</button>
</form>
</div>
</div>
<div class="col-sm-4">
</div>
</div>
```

```
</div>
</body>
</html>
```

Explanation of the Form.php codes

1. The HTML form is created with Bootstrap styling, featuring input fields for first name, last name, email, password, and confirm password.
2. PHP variables ($set_firstName, $set_lastName, $set_email) are used to retain valid input values after submission.
3. Validation error messages are displayed for each input field ($fnameErr, $lnameErr, $emailErr, $passErr, $cpassErr).
4. The form action is set to the current PHP script (htmlspecialchars($_SERVER["PHP_SELF"])) for form submission.
5. Upon successful submission, a success message ($valid) is displayed.
6. The form includes a “Register Now” button with the name attribute set to “register” for form processing.
7. Bootstrap and jQuery libraries are linked for styling and interactive features.

2. PHP Validation Script

This PHP script validates a registration form, checking first name, last name, email, password, and confirm password using regular expressions. If all validations pass, a success message is set, and input values are sanitized; otherwise, error messages are displayed, and input values are retained for correction.

File Name – script.php

```
<?php
// by default, error messages are empty
$valid=$fnameErr=$lnameErr=$emailErr=$passErr=$cpassErr='';
// by default,set input values are empty
$set_firstName=$set_lastName=$set_email='';
extract($_POST);
if(isset($_POST['register']))
{
//input fields are Validated with regular expression
$validName="/^[a-zA-Z ]*$/";
$validEmail="/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/";
```

```
$uppercasePassword = "/(?=.*?[A-Z])/";
$lowercasePassword = "/(?=.*?[a-z])/";
$digitPassword = "/(?=.*?[0-9])/";
$spacesPassword = "/^$|\s+/";
$symbolPassword = "/(?=.*?[#?!@$%^&*-])/";
$minEightPassword = "/.{8,}/";
// First Name Validation
if(empty($first_name)){
$fnameErr="First Name is Required";
}
else if (!preg_match($validName,$first_name)) {
$fnameErr="Digits are not allowed";
}else{
$fnameErr=true;
}
// Last Name Validation
if(empty($last_name)){
$lnameErr="Last Name is Required";
}
else if (!preg_match($validName,$last_name)) {
$lnameErr="Digits are not allowed";
}
else{
$lnameErr=true;
}
//Email Address Validation
if(empty($email)){
$emailErr="Email is Required";
}
else if (!preg_match($validEmail,$email)) {
$emailErr="Invalid Email Address";
}
else{
```

```
$emailErr=true;
}
// password validation
if(empty($password)){
$passErr="Password is Required";
}
elseif (!preg_match($uppercasePassword,$password) || !preg_match($lowercasePassword,$password) ||
!preg_match($digitPassword,$password) || !preg_match($symbolPassword,$password) ||
!preg_match($minEightPassword,$password) || preg_match($spacesPassword,$password)) {
$passErr="Password must be at least one uppercase letter, lowercase letter, digit, a special character with no spaces
and minimum 8 length";
}
else{
$passErr=true;
}
// form validation for confirm password
if($cpassword!=$password){
$cpassErr="Confirm Password doest Matched";
}
else{
$cpassErr=true;
}
// check all fields are valid or not
if($fnameErr==1 && $lnameErr==1 && $emailErr==1 && $passErr==1 && $cpassErr==1)
{
$valid="All fields are validated successfully";
//legal input values
$firstName= legal_input($first_name);
$lastName= legal_input($first_name);
$email= legal_input($email);
$password= legal_input($password);
// here you can write Sql Query to insert user data into database table
}else{
```

```
// set input values is empty until input field is invalid
$set_firstName=$first_name;
$set_lastName= $last_name;
$set_email= $email;
}
}
// convert illegal input value to ligal value formate
function legal_input($value) {
$value = trim($value);
$value = stripslashes($value);
$value = htmlspecialchars($value);
return $value;
}
?>
```

Explanation –

1. The PHP script initializes error messages and default values for input fields.
2. Input values are extracted from the POST request using extract($_POST).
3. The script validates first name, last name, email, password, and confirm password using regular expressions.
4. Validation errors are assigned to corresponding error variables ($fnameErr, $lnameErr, etc.).
5. If all validations pass, a success message ($valid) is set, and input values are sanitized using the legal_input function.
6. The legal_input function trims, strips slashes, and converts special characters to HTML entities for input sanitization.
7. If any validation fails, the input values are set to the provided values to maintain user input for correction.

How can I sanitize user input with PHP?

There are several ways to sanitize user input in PHP, depending on the type of data and how you plan to use it. Here are some common methods:

Use htmlspecialchars() to encode special characters in HTML. This is useful when you want to display user input as HTML and want to prevent script injection attacks.

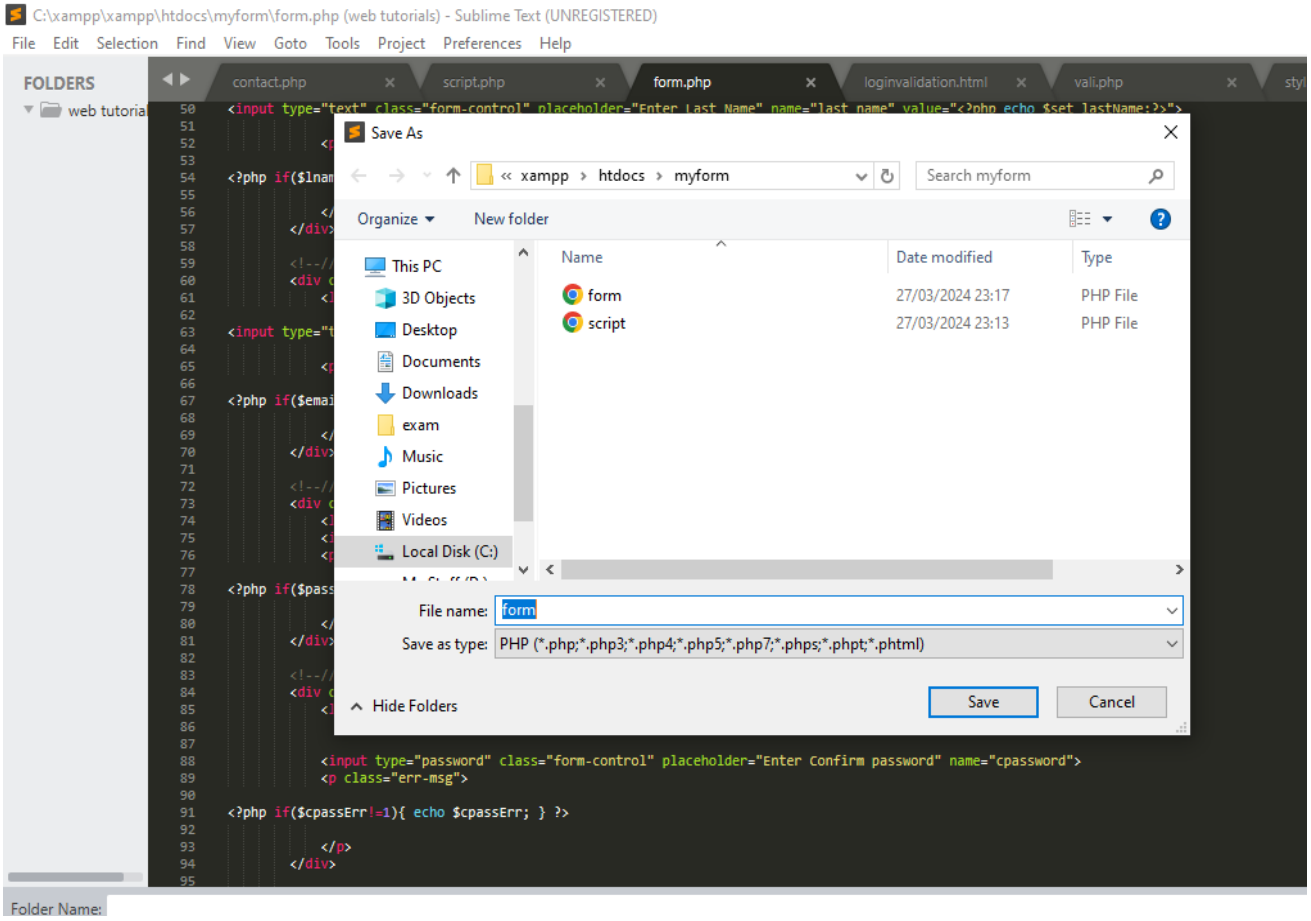
Use strip_tags() to remove HTML and PHP tags from a string. This is useful when you want to allow users to format their input with basic HTML tags, but want to remove potentially malicious tags.

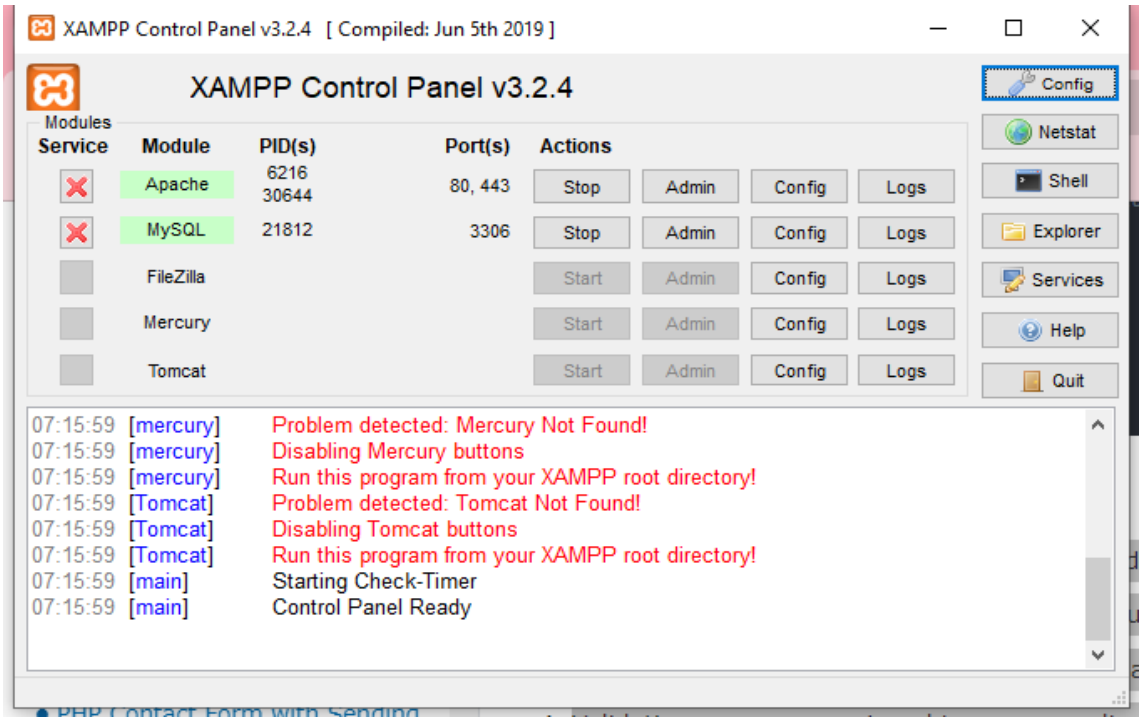
Use trim() to remove leading and trailing whitespace from a string. This is useful for cleaning up user input, especially when you are expecting a specific format (e.g. a username or password).

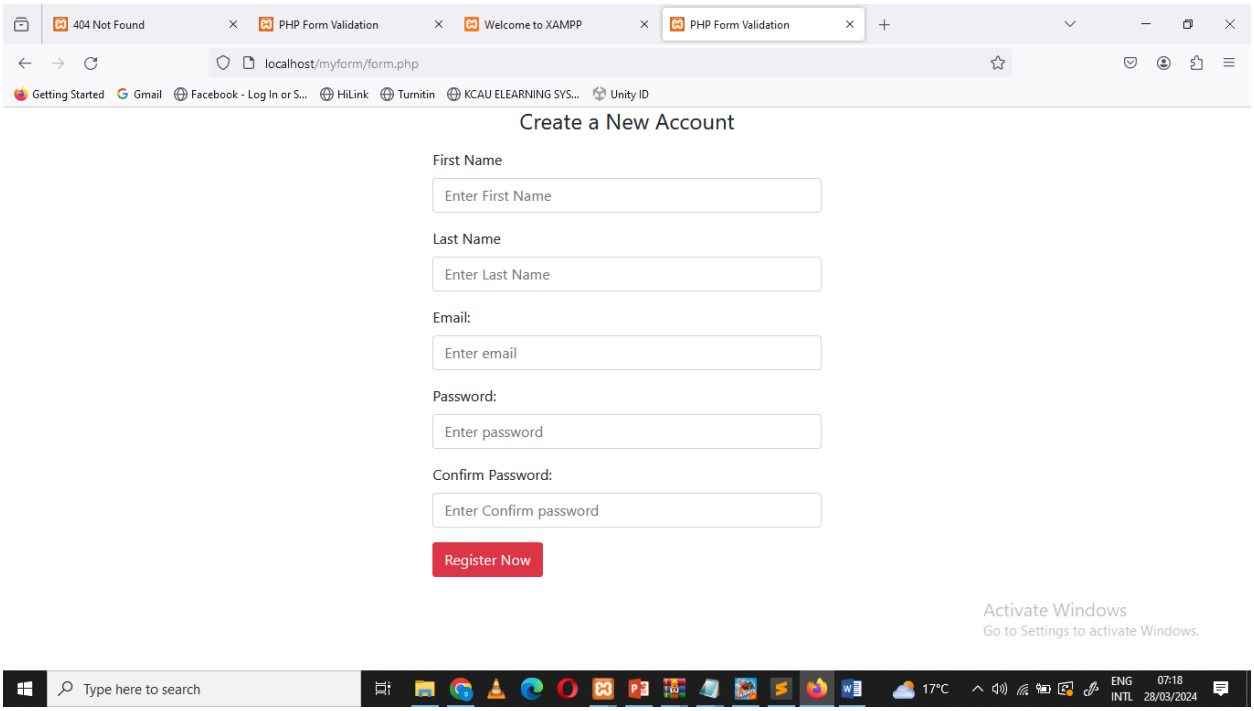
Use addslashes() to add backslashes to a string. This is useful when you are storing user input in a database and want to prevent SQL injection attacks.

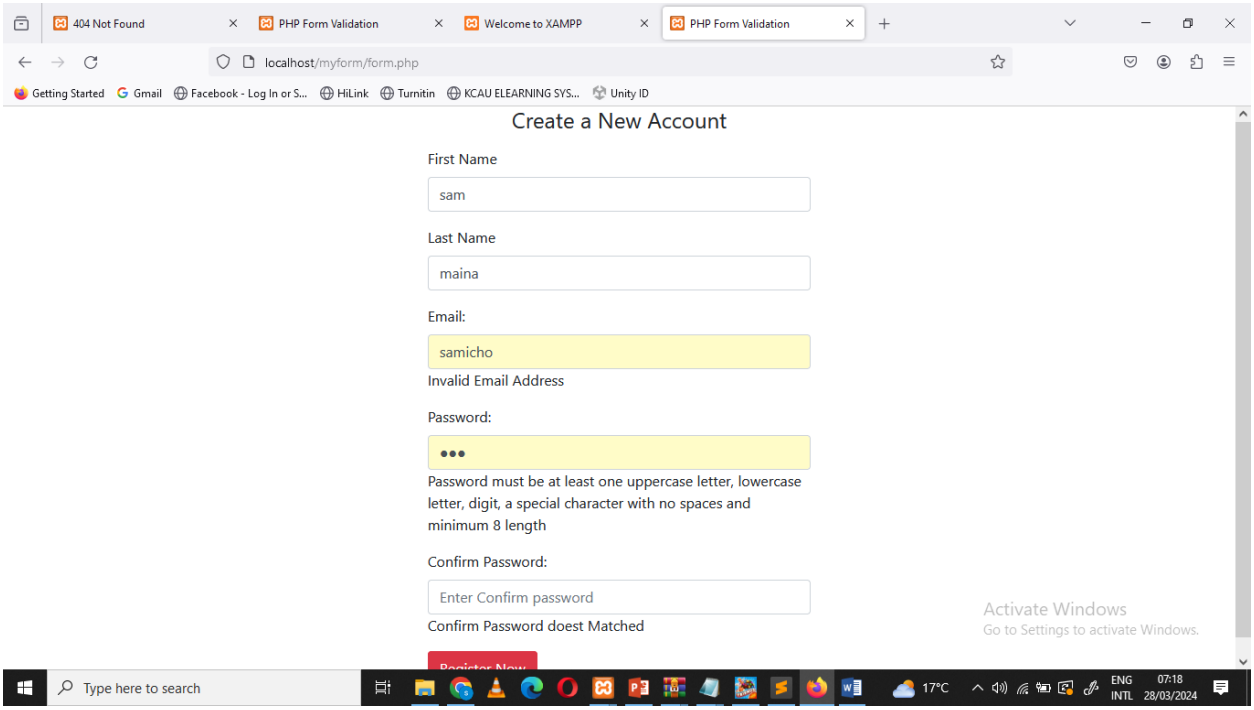
Use filter_var() with a specific filter flag to sanitize user input. For example, you can use FILTER_SANITIZE_EMAIL to remove all illegal characters from an email address.

Out put


C:\xampp\xampp\htdocs\myform\form.php (web tutorials) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
FOLDERS
contact.php
script.php
form.php
loginvalidation.html
vali.php
Save As
xampp › htdocs › myform
Search myform
Organize
New folder
This PC
3D Objects
Desktop
Documents
Downloads
exam
Music
Pictures
Videos
Local Disk (C:)
Name
Date modified
Type
form
27/03/2024 23:17
PHP File
script
27/03/2024 23:13
PHP File
File name: form
Save as type: PHP (*.php;*.php3;*.php4;*.php5;*.php7;*.phps;*.phpt;*.phtml)
Hide Folders
Save
Cancel
Folder Name:


XAMPP Control Panel v3.2.4 [Compiled: Jun 5th 2019]
XAMPP Control Panel v3.2.4
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
6216
30644
80, 443
Stop
Admin
Config
Logs
MySQL
21812
3306
Stop
Admin
Config
Logs
FileZilla
Start
Admin
Config
Logs
Mercury
Start
Admin
Config
Logs
Tomcat
Start
Admin
Config
Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
07:15:59 [mercury] Problem detected: Mercury Not Found!
07:15:59 [mercury] Disabling Mercury buttons
07:15:59 [mercury] Run this program from your XAMPP root directory!
07:15:59 [Tomcat] Problem detected: Tomcat Not Found!
07:15:59 [Tomcat] Disabling Tomcat buttons
07:15:59 [Tomcat] Run this program from your XAMPP root directory!
07:15:59 [main] Starting Check-Timer
07:15:59 [main] Control Panel Ready





Email section

We have invalid error message validation because

Some of Validation rules for field

Field	Validation Rules
Name	Should required letters and white-spaces

Email	Should required @
Website	Should required a valid URL
Radio	Must be selectable at least once
Check Box	Must be checkable at least once
Drop Down menu	Must be selectable at least once

How can I sanitize user input with PHP?

There are several ways to sanitize user input in PHP, depending on the type of data and how you plan to use it. Here are some common methods:

Use htmlspecialchars() to encode special characters in HTML. This is useful when you want to display user input as HTML and want to prevent script injection attacks.

Use strip_tags() to remove HTML and PHP tags from a string. This is useful when you want to allow users to format their input with basic HTML tags, but want to remove potentially malicious tags.

Use trim() to remove leading and trailing whitespace from a string. This is useful for cleaning up user input, especially when you are expecting a specific format (e.g. a username or password).

Use addslashes() to add backslashes to a string. This is useful when you are storing user input in a database and want to prevent SQL injection attacks.

Use filter_var() with a specific filter flag to sanitize user input. For example, you can use FILTER_SANITIZE_EMAIL to remove all illegal characters from an email address.